

REPORT S9/L2

ANALISI STATICA PE

notepad-classico.exe — Identificazione librerie e sezioni

Ambiente: FlareVM | Strumento: CFF Explorer VIII

1. Introduzione

L'analisi statica di un file eseguibile (PE — Portable Executable) consiste nell'esaminare la struttura del file senza eseguirlo, al fine di raccogliere informazioni sul suo comportamento potenziale. Le due fonti principali di informazioni nell'analisi statica sono la **Import Directory** (librerie caricate dal programma) e i **Section Headers** (struttura interna del file PE).

Il campione analizzato è **notepad-classico.exe**, un eseguibile che si presenta come un comune blocco note Windows ma che, come vedremo, presenta numerose anomalie strutturali tipiche di software malevolo.

2. Librerie importate (Import Directory)

La Import Directory di un PE elenca le DLL (Dynamic Link Library) che il programma carica al momento dell'avvio e le funzioni che utilizza da ciascuna. Analizzarla permette di capire quali capacità ha il programma: accesso alla rete, al filesystem, al registro, ecc.

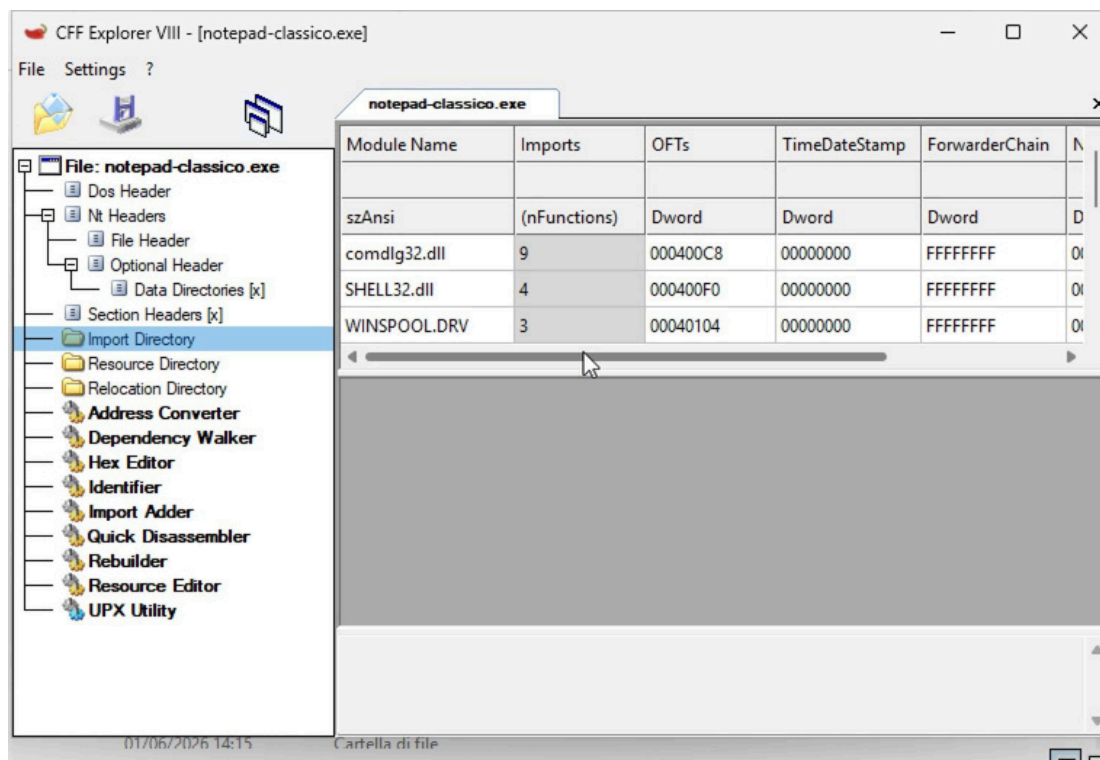


Figura 1 – Import Directory di notepad-classico.exe in CFF Explorer: solo 3 DLL importate.

DLL / Modulo	Funzioni importate	Categoria	Descrizione
comdlg32.dll	9	UI / Dialoghi	Common Dialog Box Library. Fornisce le finestre di dialogo standard di Windows: Apri file, Salva file, Stampa, Selezione font e colore. In un notepad legittimo e' attesa, ma 9 import sono molti — potrebbe includere funzioni usate per interazioni utente malevole.
SHELL32.dll	4	Shell / OS	Windows Shell API. Gestisce operazioni su file, cartelle, icone e integrazione con l'explorer. Funzioni tipiche: ShellExecute (esecuzione processi), SHGetFolderPath (percorsi di sistema). La presenza di ShellExecute e' un segnale di attenzione: puo' lanciare processi o aprire URL.
WINSPOOL.DRV	3	Stampa	Driver del gestore di stampa di Windows (Print Spooler). Fornisce API per interagire con le stampanti. In un notepad classico e' plausibile, ma combinata con le altre due sole librerie risulta sospetta: un eseguibile legittimo importa molte piu' DLL di sistema.

■ **Attenzione:** un notepad legittimo di Windows importa tipicamente 6-8 librerie (kernel32.dll, user32.dll, gdi32.dll, advapi32.dll, comctl32.dll...). La presenza di sole 3 DLL e l'assenza di kernel32/user32 suggerisce fortemente che le librerie critiche vengano caricate DINAMICAMENTE a runtime tramite LoadLibrary/GetProcAddress, tecnica classica per eludere l'analisi statica.

2.1 Librerie caricate dinamicamente

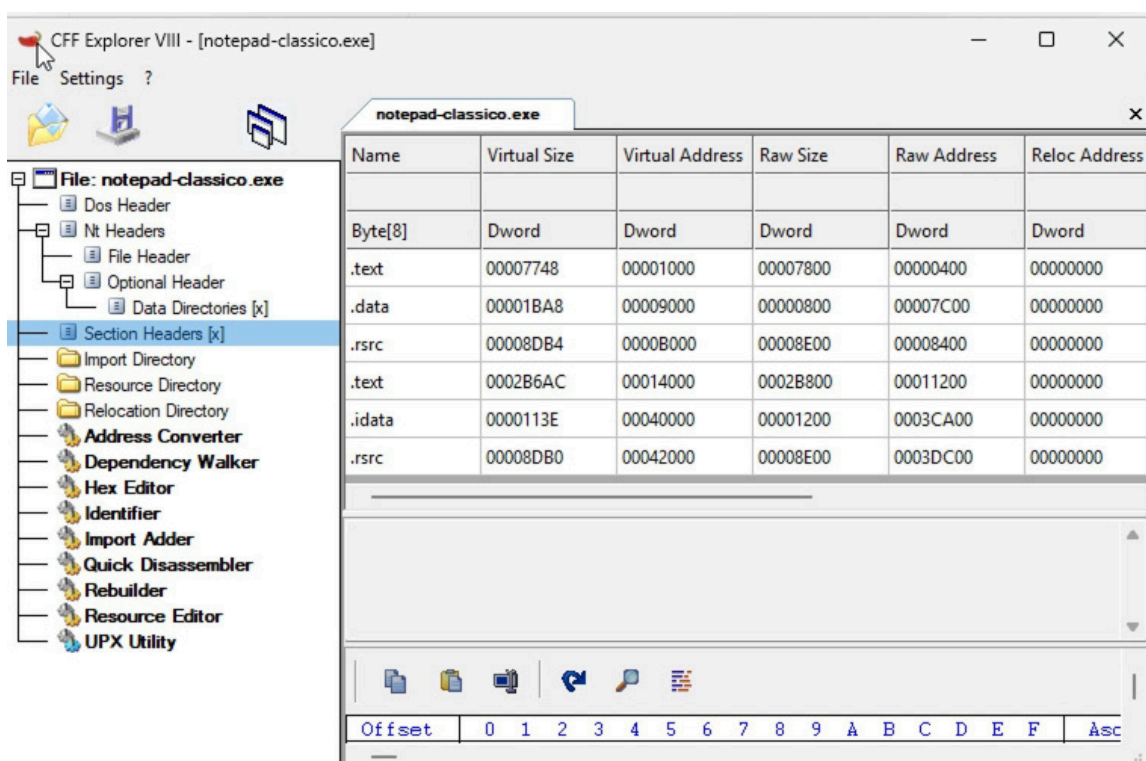
Nei malware avanzati, le funzioni più pericolose non compaiono nell'Import Directory perché vengono risolte a runtime. Il pattern tipico è:

- **LoadLibraryA/W** — carica una DLL in memoria a runtime
- **GetProcAddress** — ottiene l'indirizzo di una funzione dalla DLL caricata
- Questo permette di nascondere import come `CreateRemoteThread`, `VirtualAlloc`, `WriteProcessMemory` — funzioni tipiche di process injection e shellcode execution.

Per identificare le librerie caricate dinamicamente: usare `strings.exe` sul binario cercando nomi di DLL (.dll) e funzioni di Windows API, oppure eseguire l'analisi dinamica con un debugger (x64dbg) e intercettare le chiamate a `LoadLibrary`.

3. Sezioni del file PE (Section Headers)

I Section Headers descrivono come il file PE è suddiviso in sezioni. Ogni sezione ha un nome, una dimensione su disco (Raw Size) e una dimensione in memoria (Virtual Size). Discrepanze significative tra questi valori o nomi duplicati sono segnali di alterazione del file.



Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address
Byte[8]	Dword	Dword	Dword	Dword	Dword
.text	00007748	00001000	00007800	00000400	00000000
.data	00001BA8	00009000	00000800	00007C00	00000000
.rsrc	00008DB4	0000B000	00008E00	00008400	00000000
.text	0002B6AC	00014000	0002B800	00011200	00000000
.idata	0000113E	00040000	00001200	0003CA00	00000000
.rsrc	00008DB0	00042000	00008E00	0003DC00	00000000

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Asc

Figura 2 – Section Headers di notepad-classico.exe: 6 sezioni, di cui 2 coppie con nomi duplicati.

Nome	Virtual Size	Raw Size	Anomalia	Descrizione
.text	0x7748	0x7800	—	Sezione codice eseguibile principale. Contiene le istruzioni macchina del programma. Dimensione normale.
.data	0x1BA8	0x800	■ Sospetta	Dati inizializzati (variabili globali). Virtual Size (0x1BA8) molto piu' grande del Raw Size (0x800): spazio extra allocato a runtime, possibile area per payload o dati decrittati.
.rsrc	0x8DB4	0x8E00	—	Risorse del programma: icone, manifest, stringhe, dialoghi UI. Dimensione coerente.
.text	0x2B6AC	0x2B800	■ ANOMALIA	SECONDA sezione .text — anomalia grave. Un PE legittimo non ha mai due sezioni con lo stesso nome. Molto grande (0x2B6AC). Tecnica tipica di packer/injector per nascondere codice malevolo in una sezione aggiuntiva.
.idata	0x113E	0x1200	—	Import Address Table: contiene i puntatori alle funzioni delle DLL importate. Sezione standard.

Nome	Virtual Size	Raw Size	Anomalia	Descrizione
.rsrc	0x8DB0	0x8E00	■ ANOMALIA	SECONDA sezione .rsrc — anomalia grave. Come per .text, avere due sezioni .rsrc e' impossibile in un PE legittimo. Puo' contenere payload cifrato, shellcode o risorse sostituite dal malware.

■ **ANOMALIE CRITICHE RILEVATE:** 1) Due sezioni chiamate .text — impossibile in un PE legittimo. La seconda (.text a 0x14000) è insolitamente grande (0x2B6AC = ~177 KB) e probabilmente contiene codice iniettato o un secondo eseguibile embedded. 2) Due sezioni chiamate .rsrc — stessa anomalia. Indica manipolazione post-compilazione del PE, tecnica usata da packer e dropper per aggiungere payload cifrati.

3.1 Analisi dettagliata delle anomalie

Sezioni duplicate (.text e .rsrc):

In un PE standard, ogni sezione ha un nome univoco. La presenza di sezioni duplicate è caratteristica dei **packer** (software che comprimono/cifrano il payload e lo decomprimono a runtime) e dei **dropper** (malware che contengono ed estraggono un secondo malware). I tool di analisi comuni possono confondersi con sezioni duplicate, il che è parte della strategia evasiva.

Discrepanza Virtual Size / Raw Size in .data:

La sezione .data ha Virtual Size 0x1BA8 (~7 KB) ma Raw Size solo 0x800 (~2 KB). La differenza (circa 5 KB) viene allocata in memoria ma non esiste su disco: questo spazio extra viene spesso usato per decifrare e caricare payload a runtime, oppure come area BSS (variabili non inizializzate usate come buffer).

4. Considerazione finale — profilo del malware

Combinando le informazioni raccolte dall'analisi statica con le tecniche di analisi AI, è possibile costruire un profilo iniziale di **notepad-classico.exe**:

Mascheramento legittimo:	Il nome e l'icona del file imitano il Blocco Note di Windows per ingannare l'utente. Le librerie comdlg32 e WINSPOOL.DRV rinforzano questa apparenza di legittimità.
Evasione analisi statica:	L'assenza di librerie fondamentali (kernel32, user32) nell'Import Directory indica caricamento dinamico. Questo rende inutile una semplice scansione degli import per capire le vere capacità del malware.
Struttura PE manipolata:	Le due coppie di sezioni duplicate (.text e .rsrc) sono una firma quasi certa di packer o dropper. Il file probabilmente contiene un secondo eseguibile o shellcode cifrato che viene decompresso a runtime.
Classificazione probabile:	In base alle evidenze, notepad-classico.exe è probabilmente un dropper o un loader: un malware il cui scopo principale è scaricare/estrarre ed eseguire un payload secondario più pericoloso (RAT, ransomware, infostealer).
Prossimo passo consigliato:	Analisi dinamica con x64dbg o Cuckoo Sandbox per osservare quali DLL vengono effettivamente caricate a runtime, quali processi vengono creati e quali connessioni di rete vengono aperte.

Conclusione AI: notepad-classico.exe è un esempio di malware che sfrutta la tecnica del 'living off the land' visually (imita un'applicazione legittima) combinata con tecniche anti-analisi avanzate (import dinamici, sezioni PE duplicate). Non va mai eseguito al di fuori di un ambiente sandbox isolato.
